Task 1: Different scenarios
Player winning the game

```
ENEMY TURN
Enemy selected:  normal
Player health: 60
Enemy health: 55

PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 60
Enemy health: 35

ENEMY TURN
Enemy selected:  normal
Player health: 40
Enemy health: 35

PLAYER TURN
Select move...
normal
You selected:  normal
Player health: 40
Enemy health: 15

ENEMY TURN
Enemy selected:  last stand
Player health: 10
Enemy health: 0

GAME OVER: Player wins.
```

Enemy winning the game

```
ENEMY TURN
Enemy selected:  normal
Player health: 20
Enemy health: 20

PLAYER TURN
Select move...
heal
You selected:  heal
Player health: 35
Enemy health: 20

ENEMY TURN
Enemy selected:  heal
Player health: 35
Enemy health: 35

PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 20
Enemy health: 5

ENEMY TURN
Enemy selected:  normal
Player health: 0
Enemy health: 5

GAME OVER: Enemy wins.
```

Result of draw

```
ENEMY TURN
Enemy selected:  last stand
Player health: 40
Enemy health: 40

PLAYER TURN
Select move...
last stand
You selected:  last stand
Player health: 25
Enemy health: 10

ENEMY TURN
Enemy selected:  special
Player health: 15
Enemy health: 15

PLAYER TURN
Select move...
heal
You selected:  heal
Player health: 30
Enemy health: 15

ENEMY TURN
Enemy selected:  last stand
Player health: 0
Enemy health: 0

DRAW.
```

Task 2: Show the results for each of the below test cases and comment on which algorithm performs better and why?

(1) Linear Search Algorithm and show the algorithm efficiency for different data sizes.

```
Linear Search Execution Time: 0.000105 seconds
```

```
Linear search execution time: 2.270962 seconds
```

(2) Binary Search Algorithm and show the algorithm efficiency for different data sizes.

```
Binary search execution time: 0.000283 seconds
```

```
Binary Search (With Sorting) Execution Time: 1.656662 seconds
```

(3) Comparing linear and binary search efficiency for a sorted list generated in the code.
When looking at the time difference for the different algorithms we can see that for the shorter data set the linear algorithm gives us the shortest time meaning it is better for small data sizes. For the larger data set, the binary algorithm is much faster than the linear algorithm.

Task 3: Show the results for each of the below test cases and comment on which sorting algorithm performs better and why?

(4) Insertion Sort Algorithm and show the sorted list for different data sizes
Large:

```
Insertion Sort Execution Time: 1.648734 seconds
```

Small:

```
Insertion Sort Execution Time: 0.015570 seconds
```

(5) Bubble Sort Algorithm and show the sorted list for different data sizes.
Large:

```
Bubble Sort Execution Time: 4.524018 seconds
```

Small:

```
Bubble Sort Execution Time: 0.041785 seconds
```

(6) Measure and compare the runtimes of Bubble Sort and Insertion Sort.
In both cases using large and small data sizes we see that Insertion Sorting algorithm is much faster than the Bubble Sorting algorithm. This is due to the nature of the bubble sort which is very unoptimised for data sets like this.

Task 4: Compare the performance for each combination of sorting and searching algorithm in this task with unsorted search algorithms you have examined in Task 2.

```
Linear search execution time: 2.101066 seconds
Binary search execution time: 0.000275 seconds
Sorted array: [11, 12, 22, 25, 64]
Sorted array: [1, 11, 12, 22, 64]
Bubble Sort Execution Time: 0.043487 seconds
Insertion Sort Execution Time: 0.016247 seconds
Binary Search (With Sorting) Execution Time: 1.651868 seconds
Linear Search Execution Time: 0.000855 seconds
```

As you can see the fastest search algorithm is the binary search algorithm. The fastest sorting is the Insertion sorting algorithm.